

Paradigmas de Programación

[Inicio](#) [Paradigmas](#) [Manejo de HTML](#) [Manejo de CSS](#) [Manejo de JavaScript](#) [Formularios](#)

1. Introducción

2. Manejo de Variables

3. Ciclos

4. Condicionales

5. Funciones

6. Enlazamiento con HTML

7. Formulario

[Descargar esta pagina](#)

Manejo de JavaScript

Introducción

JavaScript (JS) es un lenguaje de programación especializado en dar interactividad y dinamismo a las páginas web, básicamente, le dice al navegador cómo debe comportarse un sitio. Es el tercer pilar de las tecnologías web junto a HTML y CSS. En este post desarrollaremos todo lo necesario para empezar a usar JS: variables, funciones y manipulación del DOM.

Manejo de Variables

- **Var**

Es la forma antigua de declarar variables. Tiene un alcance global o de función, lo que significa que puede ser accesible desde lugares inesperados y permite volver a declarar la misma variable sin que el sistema muestre un error, lo que suele causar fallos en el código moderno.

```
var nombre = "Gabriel";  
var edad = 18;  
var esEstudiante = true;
```

- **Let**

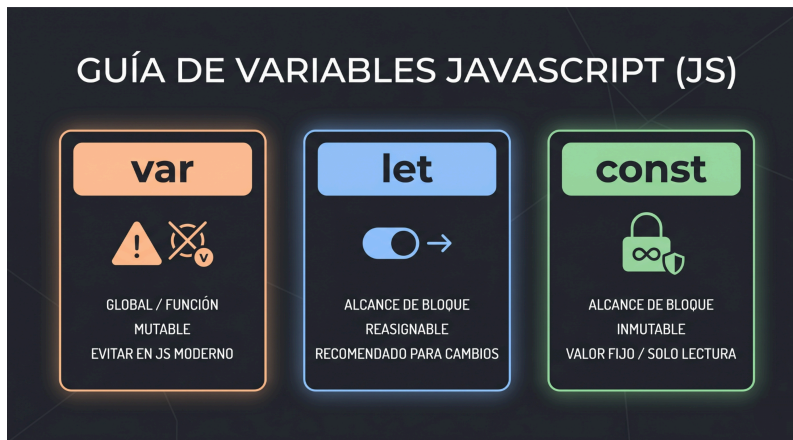
Es la forma moderna de declarar variables. Tiene un alcance de bloque, lo que significa que solo es accesible dentro del bloque donde se declara, evitando problemas de scope que pueden surgir con var.

```
let nombre = "Gabriel";  
let edad = 18;  
let esEstudiante = true;
```

- **Const**

Es la forma de declarar constantes en JavaScript. Una vez asignado un valor, no puede ser reasignado ni redeclarado.

```
const CI = 33863444;  
const NOMBRE = "Gabriel";  
const EDAD = 18;
```



Ciclos

Los ciclos son estructuras de control que permiten repetir un bloque de código varias veces. En JavaScript, existen diferentes tipos de ciclos, como el ciclo for, el ciclo while y el ciclo do-while.

- **Ciclo for**

```
for (let i = 0; i < 10; i++) {  
  console.log(i);  
}
```

- **Ciclo while**

```
let i = 0;  
while (i < 10) {  
  console.log(i);  
  i++;  
}
```

- **Ciclo do-while**

```
let i = 0;  
do {  
  console.log(i);  
  i++;  
} while (i < 10);
```

Condicionales

JavaScript también contiene estructuras condicionales. Son herramientas especializadas en la toma de decisiones dentro del código; básicamente, le dicen al navegador qué camino debe seguir según la situación.

- **Estructura if y else**

La combinación if y else funciona como una bofurcación en el camino: el if evalúa si una condición es verdadera para ejecutar un bloque de código específico, mientras que el else actúa como un respaldo automático que se ejecuta inmediatamente si esa condición resulta ser falsa.

```
let velocidad = 52;

if (velocidad > 50) {
  console.log("Velocidad alta.");
} else {
  console.log("Velocidad normal.");
}
```

- **Estructura else if**

Permite comprobar una nueva condición específica en caso de que las anteriores hayan fallado. Es ideal para encadenar múltiples evaluaciones en un orden jerárquico antes de llegar al final.

```
let nivel = 2;
if (nivel === 1) {
  console.log("Nivel inicial.");
} else if (nivel === 2) {
  console.log("Nivel intermedio.");
} else {
```

```
console.log("Nivel avanzado.");  
}
```

- **Estructura Switch**

Evalúa una única variable y compara su valor directamente con una lista de casos predefinidos. Es la alternativa más limpia y organizada para evitar acumular demasiados if y else if seguidos.

```
let nivel = 2;  
switch (nivel) {  
  case 1:  
    console.log("Nivel inicial.");  
    break;  
  case 2:  
    console.log("Nivel intermedio.");  
    break;  
  default:  
    console.log("Nivel avanzado.");  
}
```

Funciones

- **Funciones normales**

Las funciones normales se declaran utilizando la palabra clave function, tienen la ventaja de admitir hoisting y poseen su propio contexto para la palabra clave this.

```
// Declaración  
function saludar(nombre) {  
  return "Hola, " + nombre;  
}  
  
// Llamada a la función  
saludar("Jose");
```

- **Funciones flecha**

Por otro lado, las funciones flecha son una alternativa moderna que utiliza el operador `=>` para reducir la sintaxis al mínimo, permitiendo omitir las llaves y la palabra `return` si el bloque tiene una sola línea; además, heredan el contexto de `this`.

```
// Declaración
const saludarFlecha = (nombre) => "Hola,
" + nombre;

// Llamada a la función
saludarFlecha("Jose");
```

Enlazamiento con HTML

Para que el navegador ejecute JavaScript, debes conectarlo al documento HTML mediante la etiqueta `<script>`. Tienes dos formas principales:

- **Script Interno**

El código se escribe directamente en el archivo HTML. Ideal para scripts muy cortos.

```
<script>
  console.log("Hola desde el HTML");
</script>
```

- **Script Externo**

El código se guarda en un archivo `.js` aparte y se vincula usando el atributo `src`. Es la forma más limpia y recomendada.

```
<script src="codigo.js"></script>
```

Evaluación: Manejo de JavaScript

1. Las variables declaradas con ``let`` tienen un alcance de bloque.

☐ Verdadero ☐ Falso

2. Una variable declarada con ``const`` puede ser reasignada con un nuevo valor más adelante.

☐ Verdadero ☐ Falso

3. El ciclo ``do-while`` evalúa su condición al final, asegurando que el código se ejecute al menos una vez.

☐ Verdadero ☐ Falso

4. Las funciones flecha utilizan el operador ``=>`` para reducir su sintaxis.

☐ Verdadero ☐ Falso

5. Para enlazar un archivo externo de JavaScript se utiliza la etiqueta ``<link>``.

☐ Verdadero ☐ Falso

[<-Regresar al inicio](#)